

Laboratório Prático — Coordenação em Sistemas Distribuídos

Algoritmo do Valentão (Bully) — Implementação em Python

Disciplina: Sistemas Distribuídos **Linguagem:** Python 3 **Duração:** 2–3 horas

Integrantes: Alice Egg e Ruan Lucas

Objetivo geral: implementar e experimentar o algoritmo Bully utilizando processos Python e comunicação TCP, observando eleições, falhas e recuperação de processos.

1. Objetivos

- Explicar o problema de eleição de coordenador em sistemas distribuídos.
- Implementar o Algoritmo do Valentão (Bully).
- Utilizar sockets TCP para comunicação entre processos.
- Detectar a indisponibilidade do coordenador e iniciar uma eleição.
- Implementar as mensagens ELECTION, OK e COORDINATOR.
- Observar por que o processo de maior ID disponível torna-se coordenador.
- Simular a recuperação de processos e analisar novas eleições.

2. Situação-problema

Considere um sistema distribuído formado por vários processos. Cada processo possui um ID único e um dos processos atua como coordenador. O coordenador pode distribuir trabalhos, coordenar operações e manter informações de controle. Quando o coordenador deixa de responder, os processos precisam eleger um novo coordenador.

```
P1 P2 P3 P4 P5
  ↑
COORDENADOR
(maior ID)
```

A regra fundamental do Bully é: **entre os processos disponíveis, o processo com maior ID torna-se coordenador.**

3. Funcionamento do Algoritmo do Valentão

Quando um processo P percebe que o coordenador não está respondendo, P inicia uma eleição. Ele envia uma mensagem ELECTION para todos os processos com IDs maiores. Se nenhum deles responder, P vence. Se algum processo maior responder OK, P desiste e o processo maior continua a eleição.

```
P2 → P3 : ELECTION
P2 → P4 : ELECTION
P2 → P5 : ELECTION

P3 → P2 : OK
P4 → P2 : OK

P4 → todos : COORDINATOR
```

Quando um processo com ID maior recebe ELECTION, ele responde OK e assume a responsabilidade de continuar a eleição. Eventualmente, todos os processos desistem, exceto o processo ativo com maior ID.

4. Mensagens

- **ELECTION** — solicita a participação na eleição.
- **OK** — confirma que o processo está ativo e que existe um processo de ID maior.
- **COORDINATOR** — anuncia o vencedor da eleição.

5. Arquitetura do laboratório

```
P1 ID=1 porta 5001 P2 ID=2 porta 5002
P3 ID=3 porta 5003 P4 ID=4 porta 5004
P5 ID=5 porta 5005
```

Todos os processos executam na mesma máquina,
utilizando sockets TCP e portas diferentes.

Cada processo funciona simultaneamente como servidor TCP e como cliente capaz de enviar mensagens aos demais processos.

6. Estrutura do projeto

```
laboratorio-bully/
■■■ processo.py
■■■ config.py
■■■ README.md
```

7. Configuração

```
# config.py
PROCESSOS = {
    1: ("127.0.0.1", 5001),
    2: ("127.0.0.1", 5002),
    3: ("127.0.0.1", 5003),
    4: ("127.0.0.1", 5004),
    5: ("127.0.0.1", 5005),
}
```

8. Implementação do processo

```
import socket
import threading
import sys

from config import PROCESSOS

class Processo:
    def __init__(self, id_processo):
        self.id = id_processo
        self.host, self.porta = PROCESSOS[self.id]
        self.coordenador = None
        self.ativo = True
        self.elegivel = True
        self.eleicao_em_andamento = False

    def iniciar_servidor(self):
        servidor = socket.socket(
            socket.AF_INET, socket.SOCK_STREAM)
        servidor.setsockopt(
            socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        servidor.bind((self.host, self.porta))
        servidor.listen()

    print(f"[P{self.id}] aguardando conexões "
          f"na porta {self.porta}")

    while self.ativo:
        try:
            conexao, endereco = servidor.accept()
            thread = threading.Thread(
                target=self.atender, args=(conexao,))
            thread.start()
        except OSError:
            break

    def atender(self, conexao):
        try:
            mensagem = conexao.recv(1024).decode()
```

```

print(f"[P{self.id}] recebeu: {mensagem}")

if mensagem == "ELECTION":
    self.receber_eleicao(conexao)
elif mensagem.startswith("COORDINATOR"):
    partes = mensagem.split(":")
    self.coordenador = int(partes[1])
    self.eleicao_em_andamento = False
    print(f"[P{self.id}] novo coordenador: "
          f"P{self.coordenador}")
    finally:
        conexao.close()

def receber_eleicao(self, conexao):
    conexao.sendall(b"OK")
    print(f"[P{self.id}] enviou OK")

if self.elegivel:
    threading.Thread(
        target=self.iniciar_eleicao).start()

def enviar(self, id_destino, mensagem):
    host, porta = PROCESSOS[id_destino]
    try:
        cliente = socket.socket(
            socket.AF_INET, socket.SOCK_STREAM)
        cliente.settimeout(2)
        cliente.connect((host, porta))
        cliente.sendall(mensagem.encode())
        resposta = cliente.recv(1024).decode()
        cliente.close()
        return resposta
    except Exception:
        return None

```

9. Implementação da eleição

```

def iniciar_eleicao(self):
    if self.eleicao_em_andamento:
        return

    self.eleicao_em_andamento = True

    print("=" * 50)
    print(f"[P{self.id}] INICIANDO ELEIÇÃO")
    print("=" * 50)

    processos_maiores = [
        pid for pid in PROCESSOS if pid >
        self.id ]

    alguem_respondeu = False

    for pid in processos_maiores:
        print(f"[P{self.id}] enviando ELECTION para
P{pid}")    resposta = self.enviar(pid, "ELECTION")

        if resposta == "OK":
            print(f"[P{self.id}] P{pid} respondeu OK")
            alguem_respondeu = True

    if not alguem_respondeu:
        self.tornar_coordenador()
    else:
        print(f"[P{self.id}] existe processo maior
ativo.")    print(f"[P{self.id}] desistindo da
eleição.")    self.eleicao_em_andamento = False

```

10. Tornando-se coordenador

```

def tornar_coordenador(self):
    self.coordenador = self.id
    self.eleicao_em_andamento = False

    print("*** * 50)
    print(f"*** P{self.id} É O NOVO COORDENADOR
***")    print("*** * 50)

    for pid in PROCESSOS:
        if pid != self.id:
            self.enviar(

```

```
pid, f"COORDINATOR:{self.id}")
```

11. Programa principal

```
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Uso: python3 processo.py <ID>")
        sys.exit(1)

    id_processo = int(sys.argv[1])
    processo = Processo(id_processo)

    servidor = threading.Thread(
        target=processo.iniciar_servidor,
        daemon=True)
    servidor.start()

    while True:
        comando = input(f"P{id_processo}] comando> ")

        if comando == "eleicao":
            processo.iniciar_eleicao()

        elif comando == "status":
            print(f"ID: P{processo.id}")
            print(f"Coordenador: P{processo.coordenador}")

        elif comando == "sair":
            break
        else:
            print("Comandos: eleicao | status | sair")
```

12. Execução

```
python3 processo.py 1
python3 processo.py 2
python3 processo.py 3
python3 processo.py 4
python3 processo.py 5
```

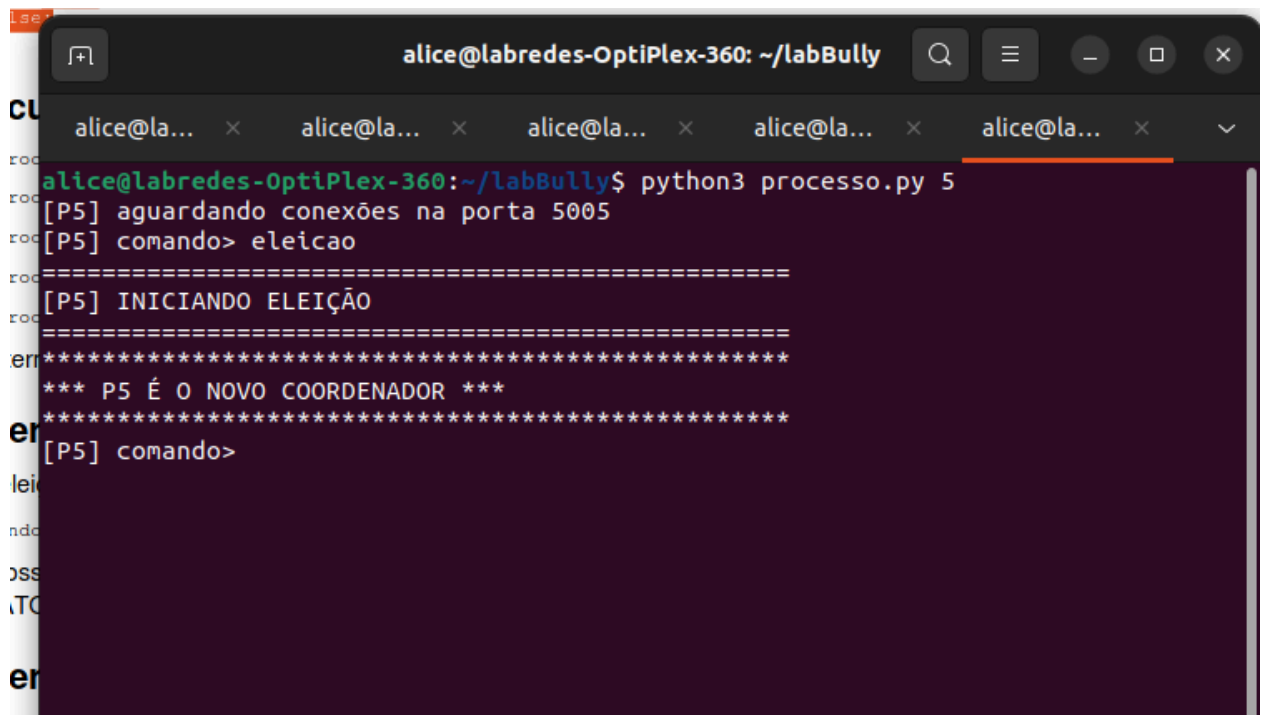
Abra cinco terminais, executando um processo diferente em cada terminal.

13. Experimento 1 — Eleição inicial

Execute a eleição no processo 5:

```
[P5] comando> eleicao
```

Como P5 possui o maior ID, não existem processos maiores para consultar. P5 torna-se coordenador e envia COORDINATOR:5 aos demais.



```

alice@labredes-OptiPlex-360: ~/labBully
alice@la... x  alice@la... x  alice@la... x  alice@la... x  alice@la... x
alice@labredes-OptiPlex-360:~/labBully$ python3 processo.py 5
[P5] aguardando conexões na porta 5005
[P5] comando> eleicao
=====
[P5] INICIANDO ELEIÇÃO
=====
*****
*** P5 É O NOVO COORDENADOR ***
*****
[P5] comando>

```

14. Experimento 2 — Falha do coordenador

Encerre P5 com o comando **sair**. Depois, no P2, execute **eleicao**. P2 consulta P3, P4 e P5. P3 e P4 respondem OK; P5 não responde. P2 desiste. P3 consulta P4; P4 responde. P4 finalmente consulta P5, não recebe resposta e torna-se coordenador.

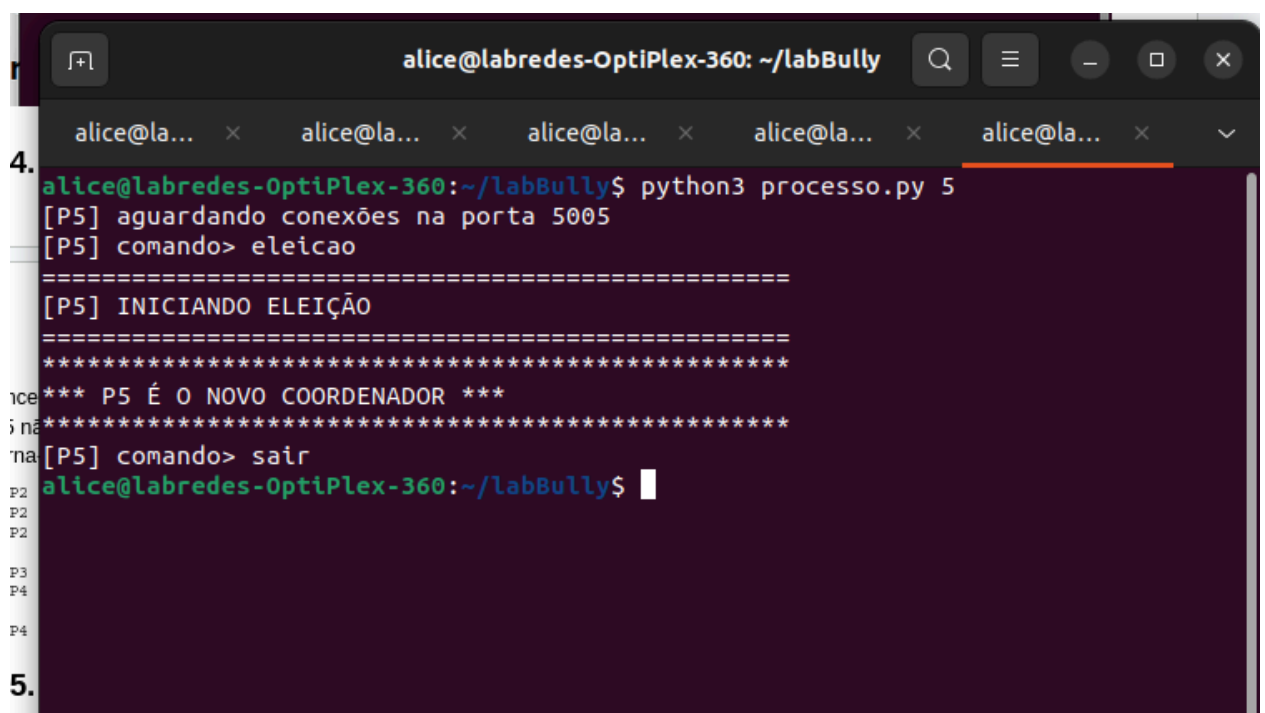
```

P2 → P3 : ELECTION P3 → P2 : OK
P2 → P4 : ELECTION P4 → P2 : OK
P2 → P5 : ELECTION P5 : falhou

P3 → P4 : ELECTION P4 → P3 : OK
P4 → P5 : ELECTION P5 : falhou

P4 → todos : COORDINATOR:4

```



```

alice@labredes-OptiPlex-360: ~/labBully
alice@la... x  alice@la... x  alice@la... x  alice@la... x  alice@la... x
4. alice@labredes-OptiPlex-360:~/labBully$ python3 processo.py 5
[P5] aguardando conexões na porta 5005
[P5] comando> eleicao
=====
[P5] INICIANDO ELEIÇÃO
=====
*****
*** P5 É O NOVO COORDENADOR ***
*****
[P5] comando> sair
alice@labredes-OptiPlex-360:~/labBully$

```

```

alice@labredes-OptiPlex-360: ~/labBully
alice@la... x  alice@la... x  alice@la... x  alice@la... x  alice@la... x
alice@labredes-OptiPlex-360:~/labBully$ python3 processo.py 2
[P2] comando> [P2] aguardando conexões na porta 5002
[P2] recebeu: COORDINATOR:5
[P2] novo coordenador: P5
[P2] eleicao
=====
[P2] INICIANDO ELEIÇÃO
=====
[P2] enviando ELECTION para P3
[P2] P3 respondeu OK
[P2] enviando ELECTION para P4
[P2] P4 respondeu OK
[P2] enviando ELECTION para P5
[P2] existe processo maior ativo.
[P2] desistindo da eleição.
[P2] comando> [P2] recebeu: COORDINATOR:4
[P2] novo coordenador: P4
[P2] recebeu: COORDINATOR:4
[P2] novo coordenador: P4

```

15. Experimento 3 — Recuperação do processo 5

Execute novamente `python3 processo.py 5` e, no P5, execute `eleicao`. Como P5 possui o maior ID, ele vence e recupera a coordenação.

```

alice@labredes-OptiPlex-360: ~/labBully
alice@la... x  alice@la... x  alice@la... x  alice@la... x  alice@la... x
alice@labredes-OptiPlex-360:~/labBully$ python3 processo.py 5
[P5] aguardando conexões na porta 5005
[P5] comando> eleicao
=====
[P5] INICIANDO ELEIÇÃO
=====
*****
*** P5 É O NOVO COORDENADOR ***
*****
[P5] comando> sair
alice@labredes-OptiPlex-360:~/labBully$ python3 processo.py 5
[P5] comando> [P5] aguardando conexões na porta 5005
[P5] eleicao
=====
[P5] INICIANDO ELEIÇÃO
=====
*****
*** P5 É O NOVO COORDENADOR ***
*****
[P5] comando> 

```

16. Experimento 4 — Recuperação de processo menor

Com P4 como coordenador, encerre P2 e depois recupere P2. Se P2 iniciar uma eleição, P4 responderá. P2 não poderá tornar-se coordenador enquanto existir um processo ativo com ID maior.

```

alice@labredes-OptiPlex-360: ~/labBully
[P4] recebeu: ELECTION
[P4] enviando ELECTION para P5
[P4] enviou OK
*****
*** P4 É O NOVO COORDENADOR ***
*****
=====
[P4] INICIANDO ELEIÇÃO
=====
[P4] enviando ELECTION para P5
*****
*** P4 É O NOVO COORDENADOR ***
*****
[P4] recebeu: COORDINATOR:5
[P4] novo coordenador: P5
eleicao
=====
[P4] INICIANDO ELEIÇÃO
=====
[P4] enviando ELECTION para P5
*****
*** P4 É O NOVO COORDENADOR ***
*****
[P4] comando>

```

```

alice@labredes-OptiPlex-360: ~/labBully
[P2] novo coordenador: P4
[P2] recebeu: COORDINATOR:5
[P2] novo coordenador: P5
[P2] recebeu: COORDINATOR:4
[P2] novo coordenador: P4
sair
alice@labredes-OptiPlex-360:~/labBully$ python3 processo.py 2
[P2] aguardando conexões na porta 5002
[P2] comando> eleicao
=====
[P2] INICIANDO ELEIÇÃO
=====
[P2] enviando ELECTION para P3
[P2] P3 respondeu OK
[P2] enviando ELECTION para P4
[P2] P4 respondeu OK
[P2] enviando ELECTION para P5
[P2] existe processo maior ativo.
[P2] desistindo da eleição.
[P2] comando> [P2] recebeu: COORDINATOR:4
[P2] recebeu: COORDINATOR:4
[P2] novo coordenador: P4
[P2] novo coordenador: P4

```

17. Experimento 5 — Várias falhas

Deixe apenas P1, P2 e P3 ativos. Inicie uma eleição em P1. A sequência esperada é P1 → P2/P3; depois P2 → P3; finalmente P3 vence.

```

alice@la... x  alice@la... x  alice@la... x  alice@la... x  alice@la... x
[P1] novo coordenador: P5
[P1] recebeu: COORDINATOR:4
[P1] novo coordenador: P4
[P1] recebeu: COORDINATOR:4
[P1] novo coordenador: P4
[P1] recebeu: COORDINATOR:4
[P1] novo coordenador: P4
[P1] recebeu: COORDINATOR:4
[P1] novo coordenador: P4
eleicao
=====
[P1] INICIANDO ELEIÇÃO
=====
[P1] enviando ELECTION para P2
[P1] P2 respondeu OK
[P1] enviando ELECTION para P3
[P1] P3 respondeu OK
[P1] enviando ELECTION para P4
[P1] enviando ELECTION para P5
[P1] existe processo maior ativo.
[P1] desistindo da eleição.
[P1] comando> [P1] recebeu: COORDINATOR:3
[P1] novo coordenador: P3
[P1] recebeu: COORDINATOR:3
[P1] novo coordenador: P3

```

18. Desafio — Detecção automática de falha

Depois de implementar a eleição manual, acrescente PING/PONG para verificar periodicamente se o coordenador responde. Quando o coordenador não responder, o processo deverá iniciar automaticamente uma nova eleição.

```

P2 ■■■ PING ■■■→ P4

P2 ←■■ PONG ■■■■ P4

P2 ■■■ PING ■■■→ P4
X
falhou
↓
iniciar eleição

```

Windsurf: Refactor | Explain | Generate Docstring | X

```

def atender(self, conexao):
    try:
        mensagem = conexao.recv(1024).decode()
        print(f"[P{self.id}] recebeu: {mensagem}")

        if mensagem == "PING":
            conexao.sendall(b"PONG")

        elif mensagem == "ELECTION":
            self.receber_eleicao(conexao)

        elif mensagem.startswith("COORDINATOR"):
            partes = mensagem.split(":")
            self.coordenador = int(partes[1])
            self.eleicao_em_andamento = False
            print(f"[P{self.id}] novo coordenador: P{self.coordenador}")

    finally:
        conexao.close()

```

Windsurf: Refactor | Explain | Generate Docstring | X

```

def monitorar_coordenador(self):
    while self.ativo:
        time.sleep(5)

        # Se ainda não existe coordenador, não verifica
        if self.coordenador is None:
            continue

        # O coordenador não precisa verificar a si mesmo
        if self.coordenador == self.id:
            continue

        # Evita iniciar outra eleição enquanto uma está acontecendo
        if self.eleicao_em_andamento:
            continue

        print(f"[P{self.id}] Enviando PING para P{self.coordenador}")

        resposta = self.enviar(self.coordenador, "PING")

        if resposta == "PONG":
            print(f"[P{self.id}] P{self.coordenador} respondeu PONG")
        else:
            print(f"[P{self.id}] Coordenador não respondeu!")
            print(f"[P{self.id}] Iniciando eleição automaticamente...")

            self.coordenador = None
            self.iniciar_eleicao()

```

[illegible][illegible]

```
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
sair
ruan@ruan-Inspiron
```

[illegible]

19. Desafio adicional — Logs

Implemente registros com horário, processo emissor, processo receptor e mensagem. Exemplo:

[21:34:12]	P2	iniciou eleição
[21:34:12]	P2	→ P3 ELECTION
[21:34:12]	P2	→ P4 ELECTION
[21:34:12]	P3	→ P2 OK
[21:34:12]	P4	→ P2 OK
[21:34:13]	P3	iniciou nova eleição
[21:34:13]	P3	→ P4 ELECTION
[21:34:14]	P4	venceu
[21:34:14]	P4	→ P1 COORDINATOR
[21:34:14]	P4	→ P2 COORDINATOR
[21:34:14]	P4	→ P3 COORDINATOR

```
[P5] INICIANDO ELEIÇÃO
=====
[08:14:43] P5 iniciou eleição
=====
*** P5 É O NOVO COORDENADOR ***
=====
[08:14:43] P5 venceu a eleição
[08:14:43] P5 → P1 COORDINATOR
[08:14:43] P5 → P2 COORDINATOR
[08:14:43] P5 → P3 COORDINATOR
[08:14:43] P5 → P4 COORDINATOR
[P5] comando> [P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
[P5] recebeu: PING
salr
ruan@ruan-Inspiron-15-3520:~/laboratorio-bully$
```

```

vwm@ruan-Inspiron-15-3520: /laboratorio-bully$ python3 processo.py 2
[P2] aguardando conexões na porta 5002
[P2] comando> [P2] recebeu: COORDINATOR:5
[P2] novo coordenador: P5
[P2] Enviando PING para P5
[P2] P5 respondeu PONG
[P2] Enviando PING para P5
[P2] P5 respondeu PONG
[P2] Enviando PING para P5
[P2] P5 respondeu PONG
[P2] recebeu: COORDINATOR:4
[P2] novo coordenador: P4
[P2] Enviando PING para P4
[P2] P4 respondeu PONG
[P2] Enviando PING para P4
[P2] P4 respondeu PONG
[P2] Enviando PING para P4
[P2] P4 respondeu PONG
[P2] Enviando PING para P4
[P2] P4 respondeu PONG
[P2] Enviando PING para P4
[P2] P4 respondeu PONG

```

[illegible]

20. Questões de análise

1. Por que o processo com maior ID tende a vencer o algoritmo Bully?

R - Porque o algoritmo dá prioridade aos processos que possuem IDs maiores. Quando um processo inicia uma eleição, ele envia mensagens para todos os processos com IDs superiores ao seu. Se algum deles responder, significa que existe um processo maior ativo, que poderá assumir a eleição. No final, o processo com maior ID que estiver funcionando se torna o coordenador.

2. Considere P1, P2, P3, P4, P5 e P6. Se P6 falhar e P2 iniciar uma eleição, quais processos receberão ELECTION?

R - O P2 enviará mensagens ELECTION para P3, P4, P5 e P6, pois todos possuem IDs maiores que o dele. Como P6 está desligado, não responderá. Já P3, P4 e P5 responderão OK e poderão iniciar suas próprias eleições. No final, P5 será eleito coordenador, pois é o processo ativo com maior ID.

3. Se P5 falhar e P2 iniciar a eleição, descreva a sequência esperada de mensagens.

R - Primeiro, P2 envia ELECTION para P3, P4 e P5. Os processos P3 e P4 respondem OK, mas P5 não responde porque está desligado. Depois, P3 inicia uma eleição e envia ELECTION para P4 e P5. P4 responde OK, enquanto P5 continua sem responder. Por último, P4 envia ELECTION para P5 e, como não recebe resposta, assume a coordenação. Então P4 envia COORDINATOR:4 para os demais processos, informando que é o novo coordenador.

4. Se P4 for coordenador e P5 voltar ao sistema, o que deverá acontecer se P5 iniciar uma eleição?

R - Como P5 possui o maior ID, ele não precisa consultar nenhum processo superior. Assim, quando iniciar uma eleição, assumirá a coordenação automaticamente e enviará a mensagem COORDINATOR:5 aos outros processos. Dessa forma, P4 deixa de ser o coordenador e P5 assume seu lugar.

5. Por que o algoritmo Bully pode gerar muitas mensagens quando existem vários processos?

R - Porque, quando uma eleição começa, cada processo envia mensagens para todos os outros que possuem IDs maiores. Esses processos respondem e também podem iniciar novas eleições, gerando várias mensagens ELECTION e OK. Além disso, o vencedor precisa avisar todos os outros processos por meio da mensagem COORDINATOR. Quanto maior a quantidade de processos, maior pode ser o número de mensagens trocadas, principalmente quando ocorrem várias eleições ao mesmo tempo.

21. Desafio final

Modifique o programa para apresentar automaticamente um diagrama textual da eleição, identificando o processo que iniciou a eleição, as mensagens ELECTION, respostas OK, desistências, vencedor e mensagens COORDINATOR.

```

ruan@ruan-Inspiron-15-3520:~/laboratorio-bully$ 
ruan@ruan-Inspiron-15-3520:~/laboratorio-bully$ ^C
ruan@ruan-Inspiron-15-3520:~/laboratorio-bully$ ^C
ruan@ruan-Inspiron-15-3520:~/laboratorio-bully$ python3 processo.py 5
[P5] aguardando conexões na porta 5005
[P5] comando> eleicao
=====
[P5] INICIANDO ELEIÇÃO
=====
[08:21:18] P5 iniciou eleição
=====
*** P5 É O NOVO COORDENADOR ***
=====
[08:21:18] P5 venceu a eleição
[08:21:18] P5 → P1 COORDINATOR
[08:21:18] P5 → P2 COORDINATOR
[08:21:18] P5 → P3 COORDINATOR
[08:21:18] P5 → P4 COORDINATOR

+--- DIAGRAMA TEXTUAL DA ELEIÇÃO ---
| Visão local do processo P5
| Iniciador desta eleição local: P5
|
| [08:21:18] P5 iniciou eleição
| |
| | v
| | [08:21:18] P5 venceu a eleição
| | |
| | | v
| | | [08:21:18] P5 → P1 COORDINATOR
| | | |
| | | | v
| | | | [08:21:18] P5 → P2 COORDINATOR
| | | | |
| | | | | v
| | | | | [08:21:18] P5 → P3 COORDINATOR
| | | | | |
| | | | | | v
| | | | | | [08:21:18] P5 → P4 COORDINATOR
| | | | | | |
| | | | | | | v
| | | | | | | [Vencedor: P5]
| | | | | | |
+---+
[P5] comando> [P5] recebeu: PING
[P5] recebeu: PING

```

22. Entrega

laboratorio-bully/
 ■■■ processo.py
 ■■■ config.py
 ■■■ README.md
 ■■■ relatorio.pdf

- Descrição do algoritmo.
- Arquitetura utilizada.
- Código desenvolvido.
- Evidências dos testes.
- Logs das eleições.
- Respostas às questões.
- Discussão sobre limitações do algoritmo Bully.

23. Critérios de avaliação

Critério	Peso
Comunicação TCP entre processos	1,5
Implementação de ELECTION	1,5
Implementação de OK	1,0
Eleição do maior ID	1,5
Mensagem COORDINATOR	1,0
Simulação de falhas	1,0
Recuperação de processos	1,0

Relatório e análise	1,5
TOTAL	10,0

24. Síntese conceitual

```

Coordenador falha
↓
Processo detecta falha
↓
Inicia ELECTION
↓
Consulta processos com ID maior
↓
Existe processo maior?
■ ■
SIM NÃO
↓ ↓
desiste vence
↓ ↓
processo COORDINATOR
maior continua
a eleição

```

Conclusão esperada: o Bully não significa simplesmente que o maior ID sempre é coordenador. O coordenador é o processo de maior ID **entre os processos atualmente disponíveis**. Quando um processo maior retorna e inicia uma eleição, ele pode assumir a coordenação.